Module 8 — Spring Data JPA & Hibernate

> Highest priority. The **N+1 problem**, **lazy vs eager**, **transactions**, and **entity lifecycle** are asked in nearly every backend Spring interview. This is where seniors are separated from juniors.

Node.js bridge: JPA/Hibernate is like a much more powerful Sequelize/TypeORM: it maps objects↔tables, tracks changes automatically (dirty checking), and has a first-level cache (persistence context) that Node ORMs mostly lack.

8.1 Hibernate Architecture & the Persistence Context

1. Why Interviewers Ask This

Understanding the persistence context explains dirty checking, first-level cache, lazy loading, and the N+1 problem — all in one model.

2. Core Concept

- **JPA** = the specification (annotations, `EntityManager`). **Hibernate** = the most common implementation. **Spring Data JPA** = a layer that generates repository implementations over JPA.
- `EntityManager` manages a **persistence context** — a first-level cache of managed entities within a transaction.
- `SessionFactory` / `EntityManagerFactory` — expensive, thread-safe, one per app. `Session` / `EntityManager` — cheap, per-transaction, not thread-safe.

3. Internal Working

```
Transaction begins -> EntityManager creates a Persistence Context (1st-level cache)
  find(id)  -> checks context first (cache hit = no SQL); else SELECT, store in context
  changes to managed entities are TRACKED (snapshots)
Transaction commit -> FLUSH: Hibernate compares snapshots -> generates UPDATE/INSERT/DELETE
                      (dirty checking) -> clears context
```

4. Memory Diagram

```
+----------------- Transaction / Persistence Context -----------------+
| managed:  User#1 (snapshot: name="A")  <- same object returned every |
|                                          find(1) in this tx         |
| you call user.setName("B")  -> no SQL yet                           |
| commit -> flush -> dirty check finds name changed -> UPDATE users... |
+---------------------------------------------------------------------+
```

5. Real Production Example

You load a `User`, set a field, and never call `save()` — it still updates on commit because of **dirty checking**. Two `findById(1)` calls in one transaction hit the DB **once** (first-level cache returns the same instance).

8.2 Entity Lifecycle (states)

```
   new User()      persist()/save()      commit/close        find()
 TRANSIENT --------> MANAGED --------> DETACHED           (managed again)
 (no id, not         (tracked in       (context closed,   merge() -> back to MANAGED
  in context)         context)          changes not tracked)
                        |
                   remove() -> REMOVED -> DELETE on flush
```

- **Transient** — new object, not associated with a context, no DB row.

- **Managed/Persistent** — tracked; changes auto-flushed (dirty checking).
- **Detached** — was managed, context closed; changes NOT tracked until `merge()`.
- **Removed** — scheduled for deletion.

Trap: `LazyInitializationException` happens when you access a lazy association on a **detached** entity (outside the transaction/session) — very common in DTO mapping after the transaction closes.

8.3 First-Level Cache · Dirty Checking

- **First-level cache** = the persistence context. Always on, per-transaction. Repeated `find(id)` in one tx = one SQL. (Second-level cache is optional, across transactions — e.g., Ehcache.)
- **Dirty checking** = at flush, Hibernate compares each managed entity to its loaded snapshot and auto-generates `UPDATE` for changed fields. No explicit `save()` needed for managed entities.

8.4 Lazy vs Eager Loading + The N+1 Problem (the #1 JPA interview topic)

Lazy vs Eager

- **LAZY** — association loaded on first access (proxy until then). Default for `@OneToMany` / `@ManyToMany`.
- **EAGER** — loaded immediately with the parent. Default for `@ManyToOne` / `@OneToOne`.
- **Best practice:** make (almost) everything **LAZY**; fetch what you need explicitly with fetch joins/entity graphs. Eager collections cause hidden joins and over-fetching.

The N+1 Problem

```
Query 1:  SELECT * FROM orders;                       -- returns N orders
Then for EACH order, lazy access to order.getCustomer():
  SELECT * FROM customers WHERE id=?    (x N)          -- N extra queries
=> 1 + N queries  (N+1)  -> catastrophic latency at scale
```

Fixes

1. **Fetch join** — `@Query("select o from Order o join fetch o.customer")` → one SQL.
2. **`@EntityGraph`** — declaratively fetch associations on a repository method.
3. **Batch fetching** — `@BatchSize(size=50)` / `hibernate.default_batch_fetch_size` → `IN (...)`.
4. **DTO projection** — select only needed columns via constructor expression/projection.

```
@EntityGraph(attributePaths = "customer")
@Query("select o from Order o where o.status = :s")
List<Order> findByStatus(@Param("s") Status s);   // single join, no N+1
```

Best Answer

> "The N+1 problem is one query to load N parents, then N more to lazily load each parent's association — 1+N round-trips that kill latency. I default associations to LAZY to avoid accidental over-fetching, then fetch exactly what I need with a `join fetch`, an `@EntityGraph`, batch fetching, or a DTO projection. I diagnose it by enabling `show-sql`/statistics and watching for repeated identical selects."

8.5 Repositories — CrudRepository, JpaRepository, Paging & Sorting

```
Repository (marker)
  CrudRepository<T,ID>        : save, findById, findAll, delete, count
    PagingAndSortingRepository: findAll(Pageable), findAll(Sort)
      JpaRepository<T,ID>      : + flush, saveAndFlush, batch, JPA-specific
```

- **Derived queries**: `findByEmailAndStatus(...)` — Spring parses the method name into a query.
- **`@Query`** — JPQL or native (`nativeQuery=true`) for complex queries.
- **Paging & Sorting**: `Page<T> findAll(Pageable)`, `PageRequest.of(page, size, Sort.by("name"))`. `Page` runs an extra count query; `Slice` doesn't (cheaper when you don't need total count).

```
interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Page<User> findByStatus(Status status, Pageable pageable);
    @Query("select u from User u where u.age > :age")
    List<User> olderThan(@Param("age") int age);
}
```

8.6 Transactions (@Transactional) · Dirty Checking

Core Concept & Internal Working

`@Transactional` is applied via an **AOP proxy** (Module 6): the proxy opens a transaction before the method, commits on normal return, and **rolls back on unchecked exceptions**. - **Default rollback:** only `RuntimeException` / `Error`. Checked exceptions do **NOT** roll back unless you set `rollbackFor = Exception.class`. (Huge trap.) - **Propagation:** `REQUIRED` (default, join or create), `REQUIRES_NEW` (suspend + new), `NESTED`, `SUPPORTS`, `MANDATORY`. - **Isolation:** `READ_COMMITTED` (typical default), `REPEATABLE_READ`, `SERIALIZABLE` — controls dirty/non-repeatable/phantom reads. - **`readOnly = true`** — optimization hint; skips dirty checking/flush for read queries.

Self-invocation trap (asked often)

Calling a `@Transactional` method from another method **in the same class** bypasses the proxy → **no transaction**. The call must come through the injected bean/proxy. Fix: move it to another bean or use self-injection.

```
@Service
class TransferService {
    @Transactional(rollbackFor = Exception.class)
    public void transfer(long from, long to, long cents) {
        accounts.debit(from, cents);
        accounts.credit(to, cents);   // if this throws, the debit rolls back
    }
}
```

Best Answer

> "`@Transactional` works through a Spring AOP proxy that begins a transaction, commits on success, and rolls back on unchecked exceptions by default — so I add `rollbackFor` for checked ones. I mark read methods `readOnly` to skip dirty checking. Two gotchas: self- invocation bypasses the proxy so the annotation is ignored, and `REQUIRES_NEW` suspends the outer transaction for an independent commit."

8.7 Relationships & Cascade Types

Mappings

- **`@OneToOne`** — user↔profile. Watch eager default; consider `@MapsId` for shared PK.
- **`@OneToMany`** / **`@ManyToOne`** — customer↔orders. Own the FK on the **`@ManyToOne`** (child) side; use `mappedBy` on the parent. Make collections LAZY.
- **`@ManyToMany`** — students↔courses via a join table. Prefer modeling the join table as its own entity when it has attributes.

Cascade Types

`PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, `ALL`. Cascade propagates operations from parent to children (e.g., saving an `Order` saves its `OrderLines`). **`orphanRemoval=true`** deletes children removed from the collection. **Trap:** `CascadeType.ALL` + `REMOVE` on a shared association can delete more than you intend.

```
@Entity
class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true,
               fetch = FetchType.LAZY)
    private List<OrderLine> lines = new ArrayList<>();

    @ManyToOne(fetch = FetchType.LAZY)     // override eager default -> avoid N+1
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Module 8 — Top 25 Interview Questions (senior answers)

1. **JPA vs Hibernate vs Spring Data JPA?** Spec vs implementation vs repository abstraction.
2. **What is the persistence context?** Per-tx first-level cache of managed entities.
3. **Entity states?** Transient, Managed, Detached, Removed.
4. **What is dirty checking?** Auto UPDATE on flush by comparing snapshots.
5. **First-level vs second-level cache?** Per-tx (always on) vs cross-tx (optional).
6. **Lazy vs eager + defaults?** Lazy (collections) vs eager (@ManyToOne/@OneToOne); prefer lazy.
7. **LazyInitializationException cause/fix?** Access lazy field after tx closed; fetch join/@EntityGraph/DTO.
8. **N+1 problem + fixes?** 1+N queries; fetch join, @EntityGraph, batch size, DTO.
9. **Fetch join vs EntityGraph?** JPQL join fetch vs declarative attributePaths.
10. **CrudRepository vs JpaRepository?** Basic CRUD vs +batch/flush/JPA features.
11. **Derived query methods?** Method name parsed into query.
12. **@Query JPQL vs native?** Entity query language vs raw SQL (`nativeQuery=true`).
13. **Paging & sorting?** `Pageable`/`Sort`; `Page` (count) vs `Slice` (no count).
14. **@Transactional internals?** AOP proxy begin/commit/rollback.
15. **Default rollback rule?** Only unchecked; add `rollbackFor` for checked.
16. **Propagation types?** REQUIRED, REQUIRES_NEW, NESTED, SUPPORTS, MANDATORY.
17. **Isolation levels?** READ_COMMITTED/REPEATABLE_READ/SERIALIZABLE (reads).
18. **readOnly benefit?** Skips dirty checking/flush.
19. **Self-invocation problem?** Same-class call bypasses proxy → no tx.
20. **Cascade types?** PERSIST/MERGE/REMOVE/REFRESH/DETACH/ALL.
21. **orphanRemoval vs REMOVE?** Removes children detached from collection vs cascades delete.
22. **Who owns the FK in @OneToMany?** The @ManyToOne (child) side; parent uses mappedBy.
23. **save vs saveAndFlush?** Defer to commit vs flush immediately.

24. **Optimistic vs pessimistic locking?** `@Version` (no lock, retry) vs DB row lock.
25. **How to detect N+1 in prod?** `show-sql`, Hibernate statistics, APM traces.

Module 8 — Top Coding Questions

- Model `Customer` 1— *`Order`* —* `Product` with correct fetch/cascade.
- Fix an N+1 with a fetch join / `@EntityGraph`.
- Write a paginated + sorted repository method returning `Page<DTO>`.
- Implement optimistic locking with `@Version` and handle the conflict.
- Write a `@Transactional` money transfer with correct rollback semantics.

Module 8 — Common Follow-ups

- “Why did your UPDATE fire without calling save()?” (dirty checking.)
- “Why does this throw LazyInitializationException in the controller?” (detached entity.)
- “Why didn’t the transaction roll back on a checked exception?” (default rule.)

Module 8 — One-Page Cheat Sheet

```
JPA(spec) / Hibernate(impl) / Spring Data JPA(repos)
Persistence context = per-tx 1st-level cache; dirty checking auto-UPDATE on flush
States: Transient -> Managed -> Detached(merge) / Removed
Lazy(collections default) vs Eager(@ManyToOne/@OneToOne default) -> prefer LAZY
N+1: 1 parent query + N child queries -> fetch join / @EntityGraph / @BatchSize / DTO
LazyInitializationException = lazy access after tx closed
Repos: Crud < PagingAndSorting < JpaRepository. Page(count) vs Slice(no count)
@Transactional = AOP proxy; rollback only unchecked (rollbackFor for checked); readOnly skips flush
Self-invocation bypasses proxy -> no tx
Propagation: REQUIRED default, REQUIRES_NEW suspends. Isolation: READ_COMMITTED typical
Cascade: PERSIST/MERGE/REMOVE/ALL; orphanRemoval deletes detached children
FK owned by @ManyToOne side; parent uses mappedBy. @Version = optimistic locking
```

Module 8 — Mock Interview (answer, then continue)

1. “Explain the N+1 problem with SQL, then show me three different fixes and their trade-offs.”
2. “I loaded an entity, changed a field, never called save — it still updated. Why?”
3. “A checked exception was thrown mid-transaction but the DB kept the partial write. Why, and how do you fix it?”
4. “Why does accessing `order.getLines()` in the controller throw `LazyInitializationException`?”
5. “Design the JPA mapping for Order/OrderLine/Product with the right fetch and cascade settings, and justify each.”

Continue to Module 9 when ready.